A Ball-Balancing Robot to Demonstrate the Effects of 5G Delay Characteristics in Networked Control Systems

Amogh Sangamesh Kuruwatti, Ramnath Rao Bekal, Shashank Birur Manjunatha, Sujay Subhash Mudalgi
Institute of Parallel and Distributed Systems, University of stuttgart
{st189970, st189465, st188462, st191272}@stud.uni-stuttgart.de

***Abstract*—Deterministic communication is a fundamental requirement for achieving stable and predictable behavior in networked real-time control systems, particularly in distributed cyber-physical systems (CPS) where prompt transmission of sensor data and reception of actuator commands is critical. This project presents the design and implementation of a network-controlled ball balancing robot, as a test bed to explore the effects of network delay, and jitter on system performance. The platform utilizes a resistive touch sensor to obtain real-time ball positions. PID controller determines the required platform orientation to balance the ball. Inverse kinematics employing rotational transformations is used to translate platform orientation into joint positions actuated by dynamixel motors. The system enables the investigation of network-induced challenges in control loops and serves as a practical demonstration platform for analyzing nonlinear dynamics and compensator design in realistic networked control environments.**

I. Introduction

In networked real-time systems, deterministic bounds on network delay and packet delay variation (jitter) are essential prerequisites for predictable and stable control. This requirement is particularly critical in distributed cyber-physical systems (CPS), where networked sensors, actuators, and controllers rely on the timely transmission of sensor values and actuator commands to ensure a required quality of control and comply with safety requirements. Recognizing this need, standardization efforts, notably from the IEEE Time-Sensitive Networking (TSN) Task Group, have proposed mechanisms enabling deterministic communication over Ethernet (IEEE 802.3), integrated within the IEEE 802.1Q standard [1].

Recently,the transition from wired to wireless Time-Sensitive Networking is achieved, for instance, by modeling the 5G system as a logical 5G-TSN bridge [2]. From the perspective of the centralized network controller (CNC) [3],the logical 5G-TSN bridge can be configured just like a normal TSN bridge. However, the port-to-port packet delay characteristics of wired and wireless TSN bridges differ significantly. In particular, while packet delay variation (PDV) within wired TSN bridges is typically in the range of 100 nanoseconds, PDV in wireless TSN bridges can span multiple milliseconds. To practically illustrate the impact of network delay and jitter on the performance of the control system, this project focuses on the design and implementation of a ball balancing robot [4] with networked control. As an open-loop unstable system, the ball's position on the plate diverges indefinitely in response to a fixed plate angle input, making effective control strategies essential. Consequently, the ball-balancing platform serves as an ideal testbed for evaluating cyberphysical systems with critical deadlines and consequences.

The remainder of this report is structured as follows. Section II gives a comprehensive overview of the system architecture. Section III describes the sensing module, while Section IV focuses on the design and implementation of the controller for the ball-balancing robot. The actuator module and how it is integrated into the system are described in Section V. Section VI introduces the network configuration used for evaluation purposes. Section VII defines the evaluation criteria used to assess system performance. Finally, Section VIII presents experimental results.

II. System Design

A. Overview



Fig. 1. Ball balancing robot experimental rig

Figure 1 shows the ball-balancing robot. The system consists of a rectangular platform supported by three articulated arms spaced 120 degrees apart, each driven by Dynamixel servo motors. A resistive touch sensor mounted on the platform tracks the real-time position of a steel ball. Each arm comprises two rigid segments connected by a revolute joint, allowing smooth, elbow-like articulation. While the lower segment is fastened to the the servo horn, the upper segments are

attached to the platform via pivot joints, enabling controlled tilting in the pitch and roll axes. For modeling and control purposes, the system assumes a simplified configuration that excludes yaw motion, focusing solely on pitch and roll dynamics. To stabilize the platform and support rapid ball movement, it is passively damped using strategically placed weights. Actuation commands are computed via a PID controller, while inverse kinematics translate platform orientation into motor angles. Control signals are transmitted over Ethernet or internally fetched (in standalone mode) using an ESP32-based microcontroller (ESP32 Dev Board / ETH01), with supporting circuitry for motor driving and power regulation housed externally (not shown in the picture).

All mechanical components are modeled in Autodesk Fusion 360 enabling rapid design iterations while maintaining precise tolerances. Each block is modular, designed separately, allowing independent reprints, upgrades without having to redesign the entire assembly.The entire system is printed in polylactic acid (PLA), with print settings tuned for a balance of mechanical strength and surface quality. To enhance dynamic stability and suppress motor-induced vibrations and jerks, metallic counterweights are placed on the lower frame, which offers stability and smoother motion under rapid control commands.

B. System Architecture

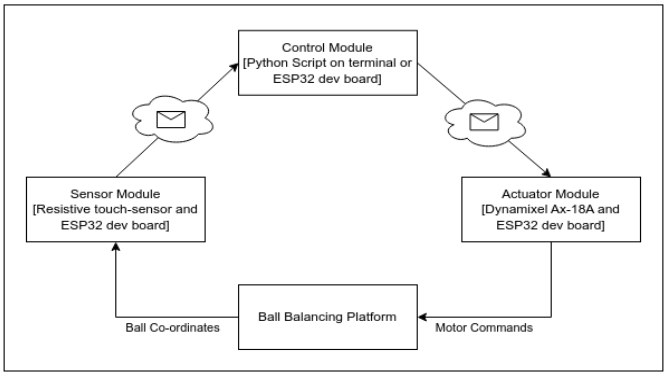


Fig. 2. System architecture

Figure 2 illustrates the general architecture of the networked ball-balancing robot system. The system can be divided into three modules. The sensing module consisting of a resistive touch sensor and ESP32 micro controller processes the data and sends the coordinates to the compute module. Compute module determines the motor angles needed to keep the ball from falling off the platform. The actuator module receives commands from the compute module and directs the servo motors to move to these positions.

The system can be operated in one of two modes. Firstly, in a distributed configuration (as illustrated above), where the sensing, control, and actuator modules are connected via Ethernet. Secondly, in a standalone configuration, all three nodes coexist on a single ESP32 based dev-board, and the network characteristics are emulated by a simple onboard ring-buffer-based emulator.

C. System Algorithm Overview

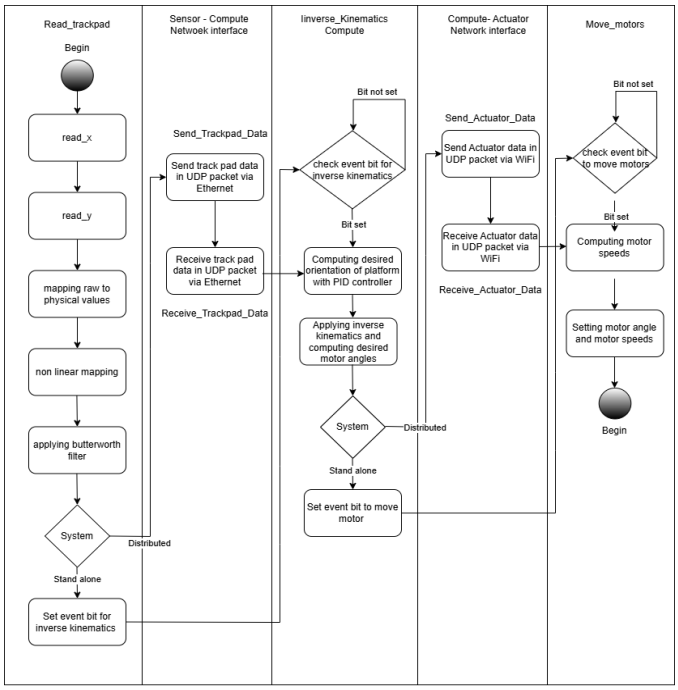


Fig. 3. System flowchart

The flow chart in Figure 3 illustrates the control flow of the system from reading the trackpad to actuating the motors. The process begins by reading x and y coordinates from the trackpad, mapped to physical values, and filtered using butter-worth filter. Depending on system configuration, if the system is in a stand alone setup, an event bit would be set for inverse kinematics computation, or else trackpad data would be sent via Ethernet in an UDP packet to a compute module. The intended motor angles are determined during the inverse kinematics computation step. The roll and pitch PID controllers determine the optimal orientation of the platform based on the ball's position on the trackpad, ensuring the ball remains stable. These orientations are then translated into the required motor angles using inverse kinematics, as described in section IV. Again based on system configuration motor angles are sent to actuator module after an event bit is set in stand alone setup or via a UDP packet to actuator node. The Move_motors block collects the data, computes the motor speed depending on the position of the ball and send the command to the motor accordingly. The architecture is flexible to both tightly coupled embedded control and distribution configuration.

III. Design of Sensing System

A 4-wire, film-to-glass touch sensor is used to read the real-time positions of the ball on the platform. The touch sensor has dimensions 187 x 141 mm, with an effective sensing

area of 172 x 129.5 mm. The resistive touchscreen comprises two electrically resistive layers, designated as the upper and lower layers, positioned facing each other and separated by a narrow gap, as seen in figure 4. The operation of resistive touch sensors or touchscreens relies on the application of mechanical pressure; upon exerting pressure on the surface, the upper layer contacts the lower layer, resulting in an alteration of electrical resistance that can be detected at the output [5]. This offers several advantages, such as cost-effectiveness,

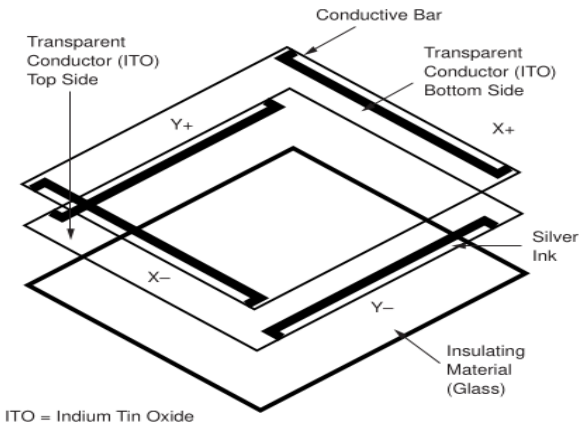


Fig. 4. 4-Wire Resistive Touch Sensor[5]

enhanced immunity to electromagnetic interference, considerable durability, compatibility with varied types of pressure-based inputs and most importantly, experimental observations during system testing indicate that the sensor is capable of reliably detecting position changes as small as 0.01mm, which is sufficient for the precision required in this application. Thus making an ideal choice for tracking the metal ball on the platform.

A. Measurement

Measuring coordinates on a trackpad involves determining the positional voltages along both the X and Y axes of the resistive touchscreen. To measure the Y-axis position, a reference voltage (+VCC) is applied to the Y+ terminal, and the Y- terminal is grounded, while the X- terminal is set to a high-impedance (tri-state) condition[6]. The resultant voltage, corresponding to the touch position along the Y-axis, is measured at the X+ terminal. Similarly, measurement of the X-axis position is performed by applying the reference voltage (+VCC) to the X+ terminal and grounding the X- terminal. Concurrently, the Y- terminal is maintained in a tri-state mode, allowing the voltage representing the touch location on the X-axis to be measured at the Y+ terminal. The resistive trackpad is interfaced using four GPIOs on the ESP32, dynamically reconfigured via software to alternate between voltage application and analog sensing.This switching is achieved using combinations of OUTPUT, INPUT_PULLDOWN, and OUTPUT_OPEN_DRAIN modes provided by the arduino framework, allowing precise analogRead() acquisition.

When mechanical pressure is applied to the panel, the resulting analog voltage readings, acquired through an analog-to-digital converter (ADC), directly correspond to the touch location along the respective axes. In order to obtain precise coordinates, a short settling time is allowed before the measurement are sampled by means of a dummy-read operation.

B. Calibration

The raw coordinates obtained are generated by the interaction between two resistive layers that form voltage gradients. When these layers are not perfectly aligned, the resulting voltage readings become systematically offset and rotated compared to the ideal coordinate system.
This necessitates the requirement of a frame of reference that represents a geometrically and physically accurate 2D space corresponding to the actual surface of the trackpad. A geometrically accurate grid which is created and placed at the back of the trackpad acts as this frame of reference.
To eliminate the inherent positional offsets in the sensor, Affine transformation (a linear mapping method that preserves points, straight lines, and parallelism) based on a multi point calibration approach is employed. It establishes a mathematical mapping from the raw sensor coordinates (x, y), acquired directly from hardware, to the corresponding actual physical coordinates (X, Y) on the reference grid. The transformation is mathematically expressed as follows:

$$X = ax + by + e \tag{1}$$

$$Y = cx + dy + f \tag{2}$$

where:

- a, b, c, d : Linear coefficients for correcting scaling, skew, and rotation effects
- e, f : Translation parameters for positional offsets

Calibration Procedure: Calibration involves identifying a set of known points on a predefined grid. For each of N calibration points i, raw sensor readings (x_i, y_i) and known trackpad coordinates (X_i, Y_i) are recorded. For each calibration point, the affine equations are:

$$X_i = a \cdot x_i + b \cdot y_i + e, \tag{3}$$

$$Y_i = c \cdot x_i + d \cdot y_i + f \tag{4}$$

A matrix is then constructed from the raw points by augmenting them with a column of ones:

$$A = \begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots \\ x_N & y_N & 1 \end{bmatrix}, \tag{5}$$

and target vectors for X and Y as:

$$X_{\text{target}} = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_N \end{bmatrix}, \tag{6}$$

$$Y_{\text{target}} = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_N \end{bmatrix} \tag{7}$$

and finally the least squares method finds parameters (i.e. the vectors $\begin{bmatrix} a & b & e \end{bmatrix}^{\mathsf{T}}$ and $\begin{bmatrix} c & d & f \end{bmatrix}^{\mathsf{T}}$) that minimize the sum of squared differences between the transformed raw readings and the target coordinates:

$$\min_{a,b,c,d,e,f} \sum_{i=1}^{N} \Big[(a\,x_i + b\,y_i + e - X_i)^2 \; + \; (c\,x_i + d\,y_i + f - Y_i)^2 \Big]$$

The obtained parameters are then used in the final transformation equations (1) and (2). With these parameters, any new reading can be transformed, correcting for offsets, rotation, and scaling issues inherent to the raw sensor data.

C. Filtering

Although allowing for a short settling time before sampling the trackpad values, the rapid rolling and sliding movement of the ball on the platform excites high-frequency components in the raw trackpad signals causing the system to twitch. To suppress these unwanted components while retaining the true, low-frequency dynamics of the ball-platform system, a second-order Butterworth low-pass filter with cutoff frequency of 5.5 Hz is applied[7].

However, application of this filter causes a group delay. Figure 5 illustrates the delay caused due to filtering. For a second-order Butterworth low-pass filter, the approximate low-frequency group delay is given by

$$\tau_g(0) \approx \frac{\sqrt{2}}{2\pi\, f_c}. \tag{8}$$

where f_c is the cut-off frequency in Hz. For the selected cut-off frequency, a delay of 40 ms is present. This sensing latency limits responsiveness of the system. To accommodate this, the control parameters(PID gains) are adjusted to compensate for the inherent sensing delay and maintain reliable behavior.

IV. Design of Control Logic

A. PID Controller(s)

The sensing node measures the instantaneous position of the ball on the resistive trackpad every $T_s = 4$ ms, yielding the raw coordinates (x_t, y_t). Defining the center of the sensing area as the setpoint $(x_0, y_0) = (0, 0)$, the errors are computed as:

$$x_{\text{err}}(t) = x_0 - x_t, \qquad y_{\text{err}}(t) = y_0 - y_t. \tag{9}$$

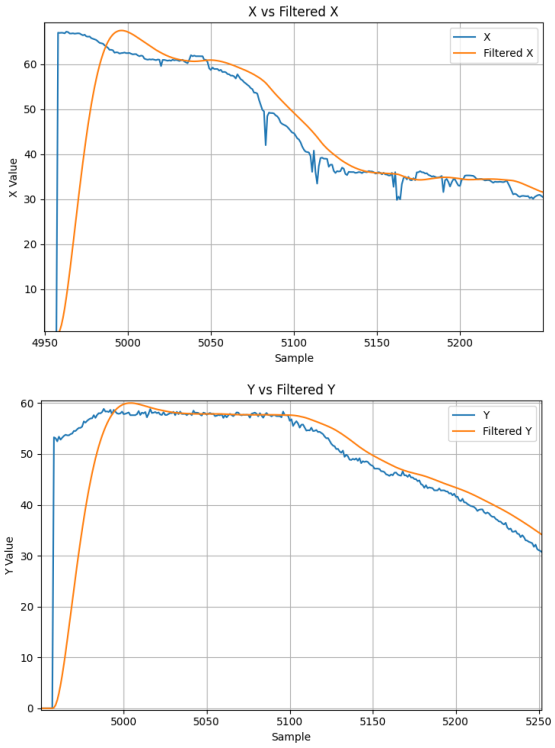


Fig. 5. Effect of filtering

With Sign Conventions:

- Positive roll $\phi > 0$: right-roll (left-up/right-down).
- Negative roll $\phi < 0$: left-roll (right-up/left-down).
- Positive pitch $\theta > 0$: head-up.
- Negative pitch $\theta < 0$: head-down.

Thus,

$$x_{\text{err}} > 0 \;\Rightarrow\; \theta_{\text{des}} < 0, \quad y_{\text{err}} > 0 \;\Rightarrow\; \phi_{\text{des}} < 0. \tag{10}$$

Proportional (P) Term: The proportional term produces a control output directly proportional to the current error:

$$\phi_P(t) = -K_p\, y_{\text{err}}(t), \tag{11}$$

$$\theta_P(t) = -K_p\, x_{\text{err}}(t). \tag{12}$$

A larger error yields a larger corrective tilt; a smaller error yields a smaller tilt.

Derivative (D) Term: The derivative term generates a control output proportional to the rate of change of error, providing damping and reducing overshoot:

$$\phi_D(t) = -K_d \frac{d}{dt}\big[y_{\text{err}}(t)\big], \tag{13}$$

$$\theta_D(t) = -K_d \frac{d}{dt}\big[x_{\text{err}}(t)\big]. \tag{14}$$

This "velocity-sensing" action reacts to rapid error changes and opposes them.

Integral (I) Term: The integral term accumulates past error,

eliminating any steady-state offset due to friction or sensor bias:

$$\phi_I(t) = -K_i \int_0^t y_{\text{err}}(\tau)\, d\tau, \tag{15}$$

$$\theta_I(t) = -K_i \int_0^t x_{\text{err}}(\tau)\, d\tau. \tag{16}$$

By integrating error over time, the I-term drives any persistent residual error to zero.

Combining all three terms gives the desired roll and pitch angles of the platform:

Roll:

$$\phi_{\text{PID}}(t) = -\Big[K_p\, y_{\text{err}}(t) + K_d\, \dot{y}_{\text{err}}(t) + K_i \int_0^t y_{\text{err}}(\tau)\, d\tau\Big]. \tag{17}$$

Pitch:

$$\theta_{\text{PID}}(t) = -\Big[K_p\, x_{\text{err}}(t) + K_d\, \dot{x}_{\text{err}}(t) + K_i \int_0^t x_{\text{err}}(\tau)\, d\tau\Big]. \tag{18}$$

B. Inverse Kinematics

The PID controller provides the roll and pitch of the platform to balance the ball. To achieve this orientation the motors are required to be actuated in a precise manner which is calculated using the inverse kinematics. The inverse kinematics of a parallel actuated system is quite complex. To simplify the implementation, the inverse kinematics of each arm should be calculated individually [10].

Using rotational matrices as described in the following, the pivot points of each arm can be calculated together elegantly and efficiently [11]. Rotational matrices are functions that can transform a point based on the roll pitch and yaw.

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix} \tag{19}$$

$$R_y(\phi) = \begin{bmatrix} \cos\phi & 0 & \sin\phi \\ 0 & 1 & 0 \\ -\sin\phi & 0 & \cos\phi \end{bmatrix} \tag{20}$$

The three pivot points of the arms are co-plane and this plane is rolled and pitched along its y and x axis respectively. The initial positions of the orange dots as shown in figure 6

Initial positions can be calculated with the function where θ is the angle of point with x-axis.

$$f(\theta) = [R\cos(\theta),\ R\sin(\theta),\ 0] \tag{21}$$

Let P be the initial positions and P' be the final rotated positions

$$\mathbf{P} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \tag{22}$$

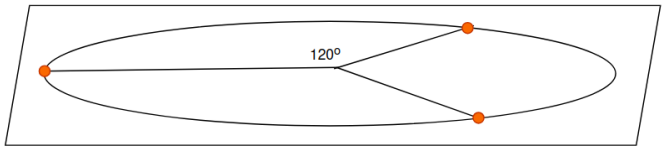


Fig. 6. Initial positions

$$\mathbf{P'} = \begin{bmatrix} x' & y' & z' \end{bmatrix} \tag{23}$$

$$\mathbf{P'} = R_x(\theta) \cdot R_y(\phi) \cdot \mathbf{P} \tag{24}$$

Arms are constrained to move in a single plane, but the above transformation does not take into account this constraint. To fix this, the points can be projected onto the constrained plane. In this system since the angles are very small the motion out of the expected plane of mechanical motion is insignificant. There is also some flexibility in the material that can account for this error.

The calculated joint in 3D coordinates is projected onto a 2D plane for the calculation of inverse kinematics. Then the coordinates are offset so that the motors are always at $(0, 0)$ and the calculated pivot position is (x, y). In the system

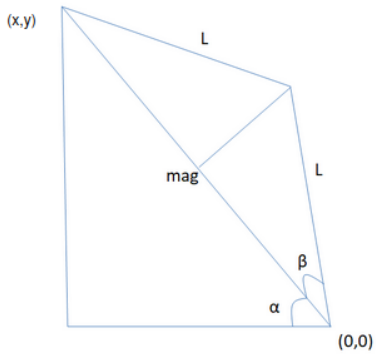


Fig. 7. Geometry of Arm

the arms are of equal length as seen in figure 7. Solving geometrically:

$$\alpha = \tan^{-1}\left(\frac{y}{x}\right) \tag{25}$$

$$\beta = \cos^{-1}\left(\frac{0.5 \cdot \text{mag}}{L}\right) \tag{26}$$

$$\text{angle} = \left(\tan^{-1}\left(\frac{y}{x}\right) + \cos^{-1}\left(\frac{0.5 \cdot \text{mag}}{L}\right)\right) \cdot \frac{180}{\pi} \tag{27}$$

The precision of the motor are 0.29 degrees and are installed at an offset of 512.

$$\text{motor_position} = 512 + \frac{180 - \text{angle}}{0.29} \tag{28}$$

V. Actuating

The Actuator module consists of an ESP32-dev board communicating with Robotis Dynamixel AX-18A servo motors, as seen in figure 8. The Dynamixel AX-18A servo is a highly efficient all-in-one solution that combines a DC motor, controller, drivers, and networking capabilities in a single package. The AX-18A utilizes a TTL serial communication protocol—Dynamixel protocol 1.0—allowing for daisy-chaining multiple servos together to create complex servo systems with fewer wires. It operates at a range of baud rates, from 7,843 bps to 1 Mbps, giving speed and flexibility in communication. It has a resolution of 0.29° deg/pulse and a step of 1 pulse/rev, offering precise control over movements; also, the servo can be controlled in terms of position, speed, and torque[8]. Robotis also provides access to the internal registers through control tables allowing precise configuration [9]. All of these advanced capabilities essential for advanced control schemes like the ball-balancing robot, where adjusting based on dynamic conditions is required.

The compute module gives the actuator ESP32-dev board

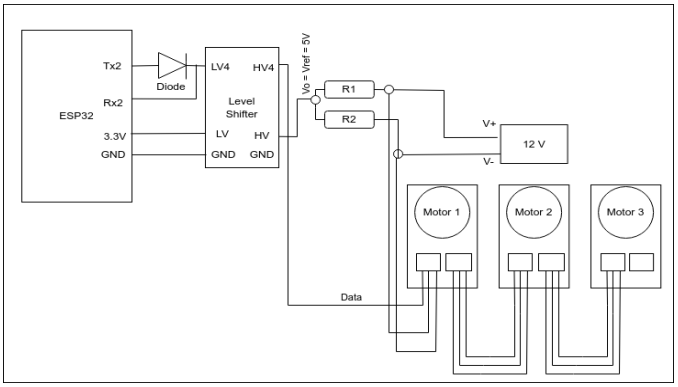


Fig. 8. Motor interface

the desired motor angles for each of the motors. This along with the required speed for each motor is communicated to the motors serially in a half duplex asynchronous manner. The diode in between the ESP32-dev board and level shifter makes sure the received data is not overwriting the transmitting data from the transmitter pin.

VI. Network

A. Network Architecture

Figure 9 shows the logical network architecture of the ball-balancing networked control system. The Sensor (left) is an ESP32 device at IP 10.1.101.125 (UDP port 9600) that periodically samples the ball's position and transmits its measurements as UDP datagrams (over Ethernet). These packets traverse the switch and are intercepted by the Compute node (top), a dedicated server at IP 10.1.101.126:9600, which executes the PID control and inverse-kinematics algorithms . The resulting motor angles are then encapsulated in UDP datagrams (sent via. Ethernet) and returned through the same switch to the Actuator ESP32 (right) at IP 10.1.101.127:9600, which drives the motors. UDP is selected as it is a connectionless protocol, which eliminates handshakes, flow-control, and retransmissions; this affords single-pass, bounded-latency delivery with minimal jitter, which is essential for real-time control systems like the ball balancing robot where timeliness supersedes absolute reliability. Reliable delivery is not strictly necessary in this system, as both sensor readings and actuator commands are transmitted periodically—newer messages naturally supersede older ones, effectively compensating for any occasional packet loss.

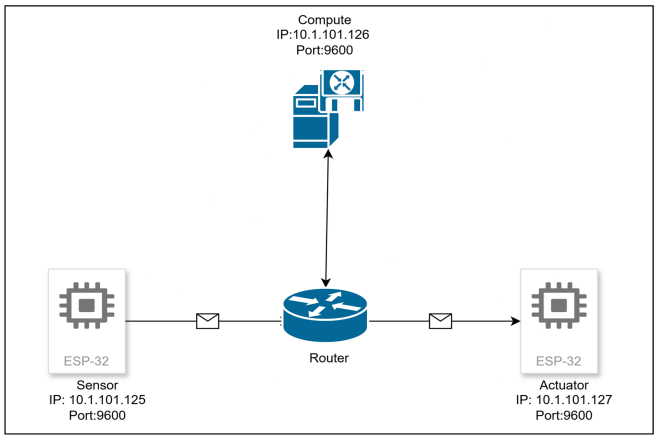


Fig. 9. Network diagram

B. 6GDetCom Emulator

To recreate the realistic port-to-port delay behavior of logical 5G TSN bridges for testing and validation of the system, 6GDetCom Emulator, a Linux-based tool designed to emulate the characteristic network delay of a 5G Time-Sensitive Networking (TSN) bridge [13] is used with up link and down-link distributions from delay histograms based on latency measurements in real 5G network provided by DETERMINISTIC6G project[14]. As seen in Figure 10, the virtual bridge connects the sensing, computing, and actuating nodes. The emulator functions as a queuing discipline (QDisc) on the egress ports of the virtual bridge. Specifically, two delay points are configured: one on the egress toward the compute module (simulating the upstream delay from sensor to controller), and another on the the actuator module (simulating the downstream delay from controller to actuator). By injecting measured or synthetic delay profiles into these paths, we can assess the resilience of the control algorithm under varying degrees of latency, jitter, and packet disorder—closely approximating the behavior of real 5G networks while maintaining full experimental control in a virtualized environment.

The 6GDetCom Emulator is evaluated with Deterministic6G

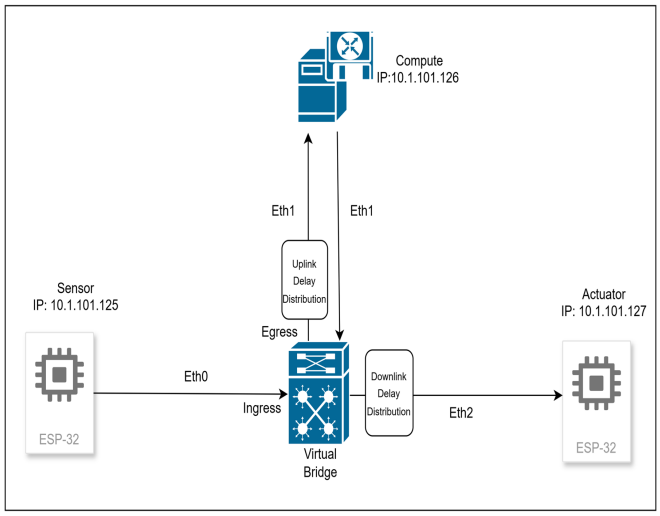


Fig. 10. Network Emulator

Measurement Data[15], which describes the packet delay (PD) of a wireless TSN bridge based on measurements from a 5G testbed in an industrial research shopfloor. Two histogram data sets are derived from measurements from the test bed, one measurement for downlink and one measurement for uplink as seen in Figures 11 and 12.

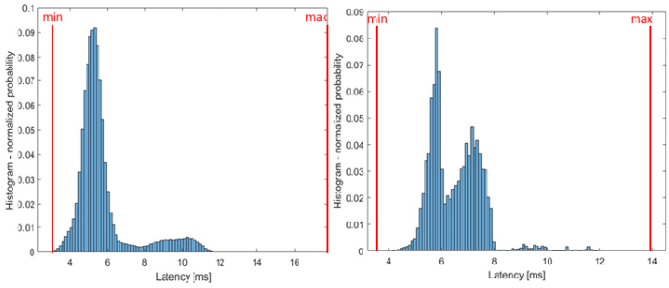


Fig. 11. Histograms of downlink and uplink dataset PD-Wireless-5G-2a

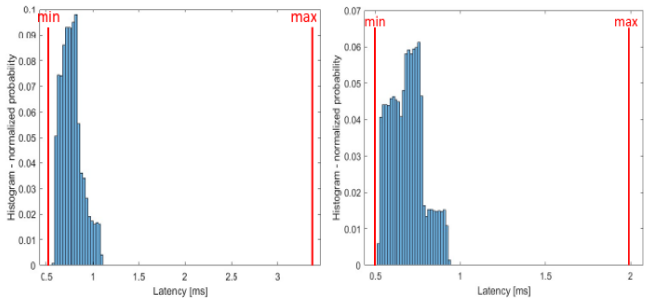


Fig. 12. Histograms of downlink and uplink dataset PD-Wireless-5G-3a

C. Ring buffer based Network emulator

The simplified network emulator implemented for real-time embedded control enables in-situ evaluation of control software under realistic network impairments - namely packet loss, delay and jitter. Outgoing "packets" are appended with a timestamped descriptor and is subjected it to stochastic loss according to a configurable probability parameter. Each descriptor is then scheduled for release after a fixed base latency augmented by a gaussian distributed jitter term[12]. A pair of lock-free, single-producer/single-consumer ring buffers maintains queued descriptors both before and after their scheduled release times, thereby reproducing the temporal characteristics of an unreliable UDP channel without reliance on external network hardware. In this project, the delay introduced by the emulator represents the total end-to-end communication latency, encompassing both uplink (sensor-to-controller) and downlink (controller-to-actuator) paths.

VII. Evaluation

A. Setup

To evaluate the effect of the network on the system, the system stability has to measured mathematically for all cases. In the context of this project, a steady state is defined as a state in which the ball's position on the platform is within $\pm 5mm$ from the center for at-least 1000 consecutive samples.
The time taken for the ball to reach the stable state from point of instigation, the transient time $T_j(\tau)$ is measured. During evaluation, the network delay τ is held constant while N statistically independent trials are executed. Each trial begins with ball being placed at an arbitrary point at t=0 and is observed for a finite horizon W. A binary settling indicator $S_j(\tau)$ is assigned to every trial $j \in \{1, \ldots, N\}$: it equals 1 if the ball reaches steady state before the horizon expires, and 0 otherwise. From these raw data three delay-specific metrics are derived: the settling probability $Ps(\tau)$, the conditional mean transient time $T_c(\tau)$, and the delay-robustness index $DRI(\tau)$.

$$\textbf{Settling indicator:} \quad S_j(\tau) = \begin{cases} 1, & \text{if } j \text{ steadies in } W, \\ 0, & \text{otherwise,} \end{cases} \tag{29}$$

$$\textbf{Settling probability:} \quad P_{\mathrm{s}}(\tau) = \frac{1}{N} \sum_{j=1}^{N} S_j(\tau). \tag{30}$$

$$\textbf{Mean settling time:} \quad \bar{T}_c(\tau) = \frac{\sum_{j=1}^{N} S_j(\tau) T_j(\tau)}{\sum_{j=1}^{N} S_j(\tau)} \tag{31}$$

B. Key Performance Indicator

$$\textbf{Delay-robustness index:} \quad \mathrm{DRI}(\tau) = P_{\mathrm{s}}(\tau) \cdot \frac{T_0}{\bar{T}_c(\tau)} \quad \text{where } T_0 \equiv \bar{T}_c(0). \tag{32}$$

The Delay-Robustness Index provides a measure of the system's tolerance to communication latency by jointly quantifying the probability of successful stabilisation and the relative degradation of transient-response speed.

Mean Squared Error:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(x_i^2 + y_i^2) \tag{33}$$

The Mean Squared Error (MSE) is computed as the average of the squared Euclidean distances of the ball's position from the center over all samples in a trial. Then, the unweighted mean of the trial-level MSE values is taken to characterize the experiment. It provides a continuous measure of the system's positional accuracy throughout the experiment, with lower values indicating more precise control and better stabilization performance.

VIII. Results

The system performance is documented and assessed using the specified KPIs for a variety of network scenarios in the section that follows. First, using an onboard ring buffer-based emulator, delay robustness and mean square error are calculated for various network properties, such as packet loss, jitter, and delay, and are recorded in table I. The system behavior for the same can be visualized in figures 13 through 20. Next, the system behavior is demonstrated with uplink and downlink distributions from delay histograms based on latency measurements in real 5G networks obtained from the DETERMINISTIC6G project. The results of which are listed in table II and visualized in figures 21 and 22.

A. Evaluation with On-board Emulator

TABLE I
Results with Onboard Emulator

D(ms)	PL(%)	J(ms)	Ps	Tc	DRI	MSE	N	FO
0	0	0	1	10.69	1	315.67	10	0
0	5	0	1	12.90	0.82	326.45	10	0
0	10	0	0.6	12.79	0.50	316.05	10	2
0	0	1	1	12.06	0.88	233.08	10	0
0	0	3	0.2	14.89	0.14	400.83	10	5
5	0	0	0.9	11.90	0.80	217.61	10	0
5	5	0	0.6	11.66	0.54	258.99	10	0
5	10	0	0.8	13.35	0.64	283.48	10	1
5	0	1	1	12.50	0.85	290.20	10	0
5	0	3	0.5	15.68	0.34	459.61	10	3
10	0	0	0.9	14.49	0.66	264.26	10	0
10	5	0	1	12.61	0.84	271.95	10	0
10	10	0	0.8	10.97	0.77	228.62	10	1
10	0	1	0.8	15.10	0.56	332.44	10	1
10	0	3	0.2	13.70	0.15	422.76	10	7
20	0	0	0.6	13.64	0.47	301.62	10	3
20	5	0	0.1	10.91	0.097	371.29	10	9
20	0	1	0.8	11.90	0.71	271.55	10	2
20	0	3	0	N.A	N.A	416.70	10	10

Abbreviations: D – Delay (ms), PL – Packet Loss (%), J – Jitter (ms), Ps – Probability of settling within 3 mm of the center in 30 s, Tc – Mean settling time, DRI – Delay Robustness Index, MSE – Mean Squared Error, N – Number of Trials, FO – Fell Off Count.

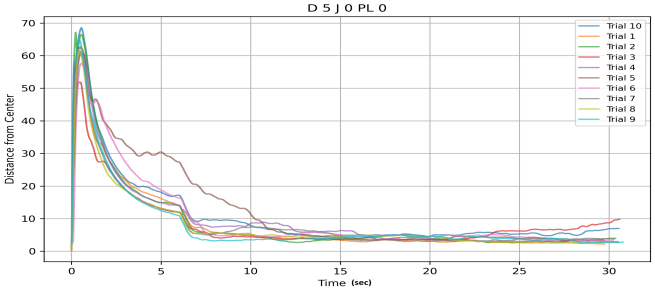


Fig. 13. Delay:5ms Jitter:0ms PacketLoss:0%

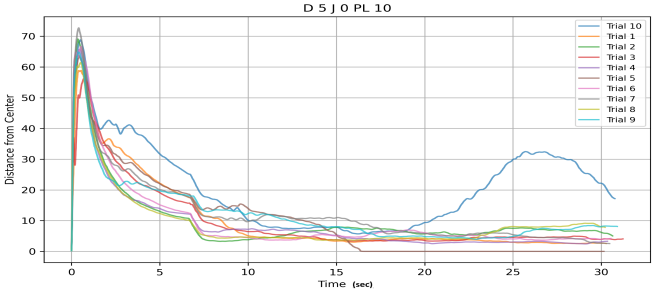


Fig. 14. Delay:5ms Jitter:0ms PacketLoss:10%

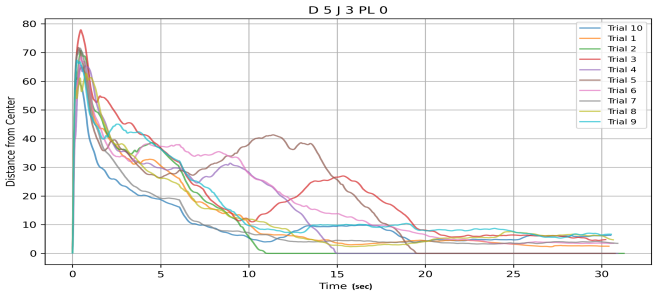


Fig. 15. Delay:5ms Jitter:3ms PacketLoss:0%

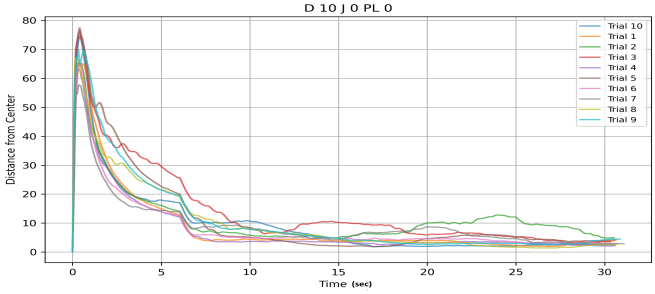


Fig. 16. Delay:10ms Jitter:0ms PacketLoss:0%

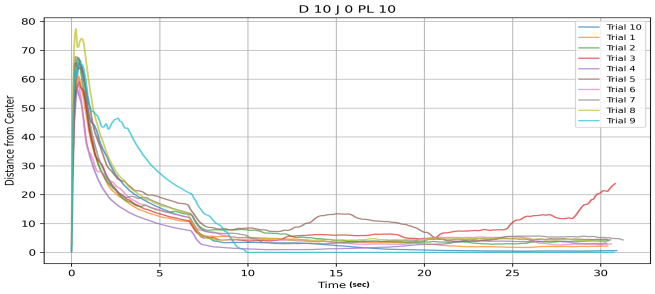


Fig. 17. Delay:10ms Jitter:0ms PacketLoss:10%

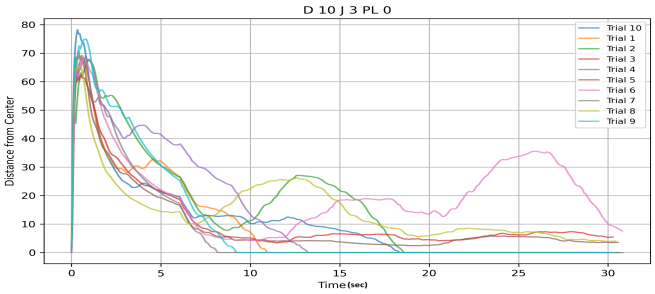


Fig. 18. Delay:10ms Jitter:3ms PacketLoss:0%

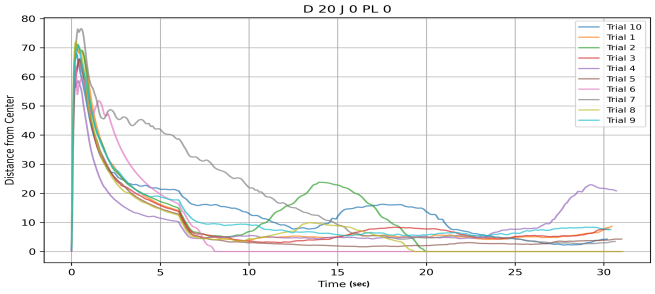


Fig. 19. Delay:20ms Jitter:0ms PacketLoss:0%

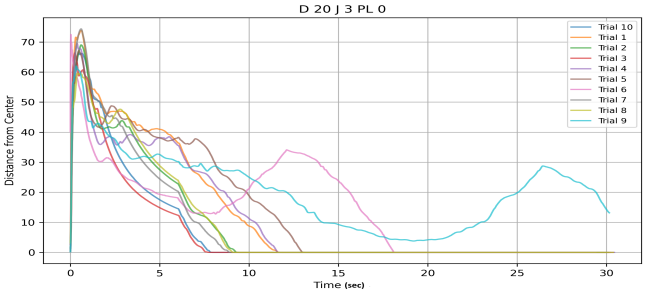


Fig. 20. Delay:20ms Jitter:3ms PacketLoss:0%

B. Evaluation with 6GDetCom Emulator

TABLE II
RESULTS WITH 6GDETCOM EMULATOR

D (ms)	PL (%)	J (ms)	Ps	Tc	DRI	MSE	N	FO
As in PD-Wireless-5G-2a Distributions			0.6	12.96	0.43	176.58	10	2
As in PD-Wireless-5G-3a Distributions			0.6	7.89	0.71	194.56	10	1

Abbreviations: D – Delay (ms), PL – Packet Loss (%), J – Jitter (ms), Ps – Probability of settling with 3 mm from the center within 30 seconds, Tc – Mean settling time for the experiment , DRI – Delay Robustness Index, MSE – Mean Squared Error, N – Number of Trials, FO – Fell Off Count, DS- Dataset[14]

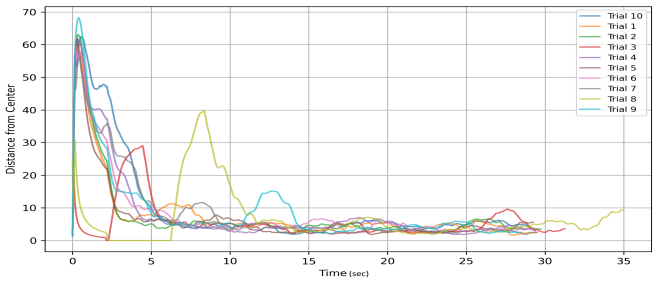


Fig. 21. Dataset 2a

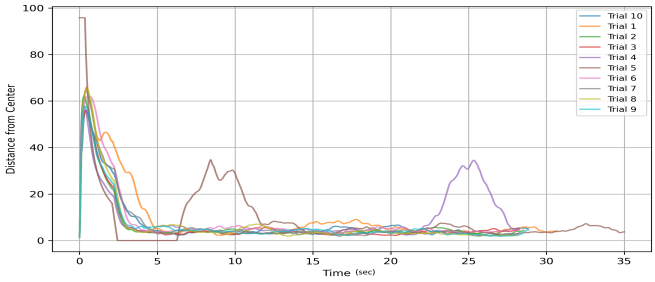


Fig. 22. Dataset 3a

C. Analysis and Interpretation

Figures 13, 16, and 19 illustrate the system's behavior under base delays of 5 ms, 10 ms, and 20 ms, respectively, without any jitter or packet loss. Each plot represents 10 experimental trials. From these, it is evident that increasing network delay results in longer times for the system to reach the defined steady state. This trend is also reflected in the gradual decrease in Delay Robustness Index (DRI) values and an increase in Mean Squared Error (MSE) across the respective experiments. As expected, delay introduces latency in the control loop, making the system respond more slowly.

A comparative analysis of Figures 13 with 14 and 16 with 17 highlights the impact of introducing packet loss, while maintaining the same base delay and jitter characteristics. In both cases, the addition of packet loss leads to a notable increase in transient durations and, in some instances, causes the ball to fall off the platform. From the tabulated data,

it can be inferred that higher base delays exacerbate the adverse effects of packet loss. This is further supported by the consistent decline in DRI values, increased MSE, and a rise in fall-off events.

Further, Figures 13 vs. 15, 16 vs. 18, and 19 vs. 20 compare scenarios where base delay and packet loss remain constant, but jitter is introduced. The presence of jitter occasionally causes the ball to deviate from its intended path momentarily or completely. This behavior is substantiated by a rise in MSE and a corresponding drop in DRI, along with an increase in the number of fall-off events across these experiments.

Lastly, evaluation using the 6GDetCom emulator reveals that although the system generally maintains a low MSE, indicating that the ball hovers near the center, it fails to stabilize it quickly and experiences occasional fall-offs. The system naturally performs better with PD-Wireless-5G-3a Distributions because of lower delay and variations, while the lower DRI for the evaluation results of the PD-Wireless-5G-2a Distributions is due to the variations in the packet delays.

Overall the effect of delay is very clear, once lower than a certain value it does not improve stability anymore but higher values does cause the system to react in a sluggish manner. There is a gradual fall off in the response of the system until a point after which the system will not stabilize. In the 6GDetCom it was evident that packet delay variations made it hard for the system stabilize the ball quickly.

While the DRI metric is particularly sensitive to variations in base delay, it is less effective in capturing the effects of jitter across different delay levels. In these scenarios, a combined assessment using both DRI and MSE offers a more thorough assessment of system performance. Furthermore, observations across both Table 1, drawn from experiments with the on-board emulator and the Deterministic-6G Dataset (2a), show that the best controller performance occurs at a network delay of 5-7 ms - indicated by lower MSE. This operating point is thought to benefit from delay-induced stabilization[16], whereby finite communication lag enhances closed-loop damping in PID-based networked control. Although the current findings are consistent with this interpretation, further analysis is needed to confirm the mechanism.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to the University of Stuttgart, Institute of Parallel and Distributed Systems (IPVS), for the opportunity and the support to undertake this project. We are especially thankful to Professor Dr. rer. nat. Frank Dürr and Mr. Simon Egger for their valuable time, insightful guidance, and continuous support throughout the course of the work. Our appreciation also extends to the Maker's Space- University of Stuttgart for providing access to essential tools, equipment, and assistance required for the 3D printing of components.

REFERENCES

[1] "IEEE Standard for Local and Metropolitan Area Networks–Bridges and Bridged Networks," in IEEE Std 802.1Q-2022 (Revision of IEEE Std 802.1Q-2018), pp.1-2163, 22 Dec. 2022, doi: 10.1109/IEEESTD.2022.10004498.

[2] 3GPP. System architecture for the 5G System (5GS). Technical Specification (TS) 23.501, 3rd Generation Partnership Project (3GPP), 09 2024. Version 18.7.0.

[3] IEEE standard for local and metropolitan area networks–bridges and bridged networks amendment 38:Configuration enhancements for time sensitive networking. IEEE Std 802.1Qdj2024 (Amendment to IEEE Std 802.1Q 2022 as amended by IEEE Std 802.1Qcz 2023, IEEE Std 802.1Qcw 2023, and IEEE Std 802.1Qcj 2023), pages 1–56, 2024

[4] A. Zeeshan, N. Nauman and M. Jawad Khan, "Design, control and implementation of a ball on plate balancing system," Proceedings of 2012 9th International Bhurban Conference on Applied Sciences & Technology (IBCAST), Islamabad, 2012, pp. 22-26, doi: 10.1109/IBCAST.2012.6177520.

[5] S. Osgood, C. Ong, R. Downs, and Burr-Brown Corporation,"TOUCH SCREEN CONTROLLER TIPS," Apr. 2000. [Online, 23/May/2025]. Available: https://www.ti.com/lit/an/sbaa036/sbaa036.pdf?ts=1744361774951

[6] "4-Wire Resistive Touch Screen Interfacing with ESP32 — ESP32," © 2018 ElectronicWings. https://www.electronicwings.com/esp32/4-wire-resistive-touch-screen-interfacing-with-esp32

[7] A. Koszewnik, K. Troc, and M. Słowik, "PID controllers design applied to positioning of ball on the Stewart platform," Acta Mechanica Et Automatica, vol. 8, no. 4, pp. 214–218, Dec. 2014, doi: 10.2478/ama-2014-0039.

[8] "Dynamixel AX-18A Smart Serial Servo," Thingbits. https://www.thingbits.net/products/dynamixel-ax-18a-smart-serial-servo

[9] "ROBOTIS e-Manual". https://emanual.robotis.com/docs/en/dxl/ax/ax-18a/

[10] R. V. V. Petrescu et al., "Inverse kinematics of a Stewart platform," Journal of Mechatronics and Robotics, pp. 45–59, Mar. 2018, doi: 10.3844/jmrsp.2018.45.59.

[11] L. Björklund and E. Gelfgren, "Stewart Platform with Inverse Kinematics: Imitation of Ocean Wave Movement using a Stewart Platform," NoviOcean, 2024.

[12] Premaratne, Upeka. (2015). Empirical network jitter measurements for the simulation of a networked control system. 2014 14th International Conference on Advances in ICT for Emerging Regions, ICTer 2014 - Conference Proceedings.

[13] L. Haug et al., "Simulating and emulating the characteristic packet delay of logical 5G TSN bridges," journal-article, 2021.

[14] 6GDetCom Network Delay Emulator, "DETERMINISTIC6G/6GDetCom_Emulator: Linux-based network delay emulator", GitHub. Available: https://github.com/DETERMINISTIC6G/6GDetCom_Emulator [Online; accessed 24-May-2025]

[15] J. Ansari, J. Sachs, Contribution of 5G Packet Delay Data Set "PD-Wireless-5G-2a", in DETERMINISTIC6G deliverable D4.1, "Digest on First DetCom Simulator Framework Release," December 2024

[16] Atay, F.M. (2009). Delay-Induced Stability: From Oscillators to Networks. In: Atay, F. (eds) Complex Time-Delay Systems. Understanding Complex Systems. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-02329-3_2